Universidad Nacional de General Sarmiento

Programación II - com3

Trabajo Práctico Integrador

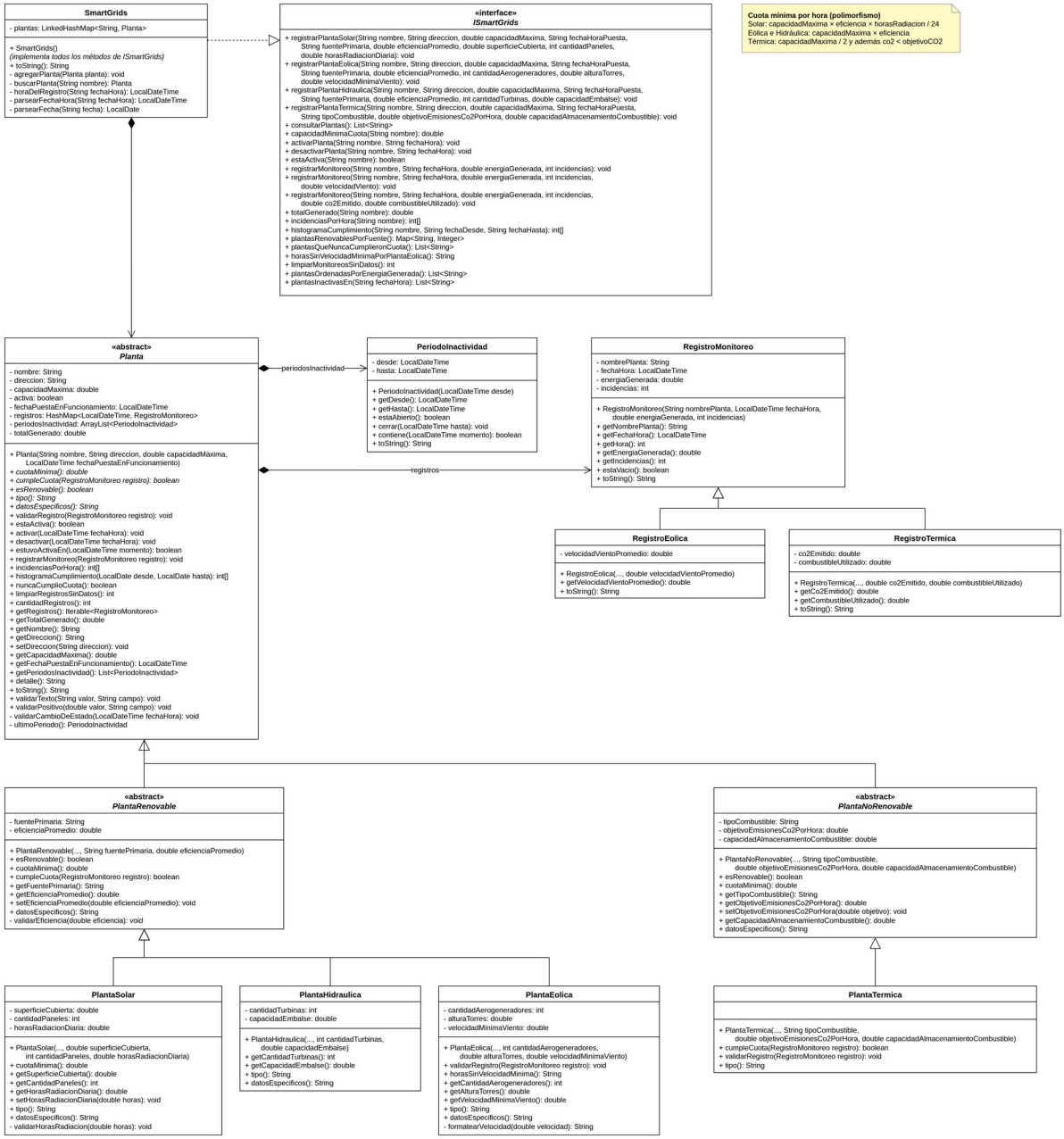
SmartGrids – Segunda parte (Implementación)

Integrantes

- Alvarez, Roberto Gonzalo
- Mazzola, Emmanuel
- Ruiz, Gonzalo

2026C2

1) Diagrama de clases



Cambios respecto de la primera parte

- **Plantas identificadas por nombre.** La interfaz `ISmartGrids` que nos dieron identifica a cada planta por su nombre, así que volvimos a usarlo como clave y quitamos los ids de plantas y de registros (`proximoIdPlanta`, `idRegistro`). `SmartGrids` guarda las plantas en un `LinkedHashMap<String, Planta>`: buscar una planta sigue siendo O(1), y por lo tanto activarla, desactivarla y consultar su estado también (requerimiento 3).
- **Fechas como texto.** La interfaz recibe las fechas como `String` (`2026-10-08T10:00:00` y `2026-10-08`). `SmartGrids` las convierte a `LocalDateTime` y

`LocalDate`; si el formato no es válido lanza `IllegalArgumentException`. Las plantas y los registros trabajan solo con `java.time`.

- **Un registro por hora.** El registro de monitoreo corresponde a una hora completa: si se informa `10:15`, se guarda como el registro de las `10:00`. Cada registro guarda el nombre de su planta (`nombrePlanta`).
- **Planta térmica sin calderas.** La interfaz no recibe los datos de las calderas al registrar una planta térmica, por lo que se quitaron de `PlantaTermica`.
- **Total generado acumulado.** Cada planta suma la energía en `totalGenerado` al registrar un monitoreo y la resta al limpiar registros. Así el requerimiento 5 es O(1) y el ordenamiento del requerimiento 12 no recorre los registros.
- **Incidencias por hora.** Se devuelven en un `int[24]`, como pide la interfaz, en lugar del mapa planteado en la primera parte.

- **Getters y setters.** Todas las clases tienen getters. Hay setters solo para los atributos que se pueden cambiar sin romper el IREP: `setDireccion`, `setEficienciaPromedio`, `setHorasRadiacionDiaria` y `setObjetivoEmisionesCo2PorHora`.
- **Errores.** Ante datos inválidos o plantas inexistentes se lanza `IllegalArgumentException`; al registrar un monitoreo en una planta inactiva se lanza `IllegalStateException`.

Requerimientos nuevos y modificados

- **1.b) Consulta de plantas.** `consultarPlantas()` devuelve el `detalle()` de cada planta, con el formato `{NOMBRE} ({ACTIVA|INACTIVA}): {FECHA_PUESTA_EN_FUNCIONAMIENTO}`.
- **10) Horas con poco viento.** Para cada planta eólica, `horasSinVelocidadMinima()` calcula la mayor velocidad registrada en cada hora del día. Se listan las horas con registros en las que esa velocidad máxima es menor que la mínima de la planta, es decir, las horas en las que nunca se alcanzó; se muestra la máxima registrada. Las horas sin registros no se listan.
- **11) Limpiar registros sin datos.** Cada planta recorre sus registros con un `Iterator` y borra con `remove()` los que tienen energía 0 e incidencias 0 (`estaVacio()`). Devuelve cuántos borró y `SmartGrids` suma lo de todas las plantas.
- **12) Plantas ordenadas por energía.** Se copian las plantas a un `ArrayList` y se ordenan con `sort` y un comparador por `totalGenerado`, de mayor a menor, con el formato `{NOMBRE_PLANTA}: {TOTAL_GENERADO}`.
- **13) Plantas inactivas en una fecha y hora.** `estuvoActivaEn(momento)` recorre los períodos de inactividad de la planta y verifica si alguno contiene el momento.

2) Conceptos de programación utilizados

- **Herencia.** `Planta` es la superclase de `PlantaRenovable` (de la que heredan `PlantaSolar`, `PlantaEolica` y `PlantaHidraulica`) y de `PlantaNoRenovable` (de la que hereda `PlantaTermica`). `RegistroEolica` y

`RegistroTermica` heredan de `RegistroMonitoreo`. Los constructores de las subclases llaman a `super` para inicializar y validar los datos comunes.

- **Clases y métodos abstractos.** `Planta`, `PlantaRenovable` y `PlantaNoRenovable` son abstractas: no existe una planta que no sea de un tipo concreto.
- **Interfaces.** En `SmartGrids` usamos la interfaz `ISmartGrids` que nos dieron. También usamos interfaces de Java: las variables y retornos se declaran como `List`, `Map`, `Iterator` y el ordenamiento del requerimiento 12 usa un `Comparator`.
- **Sobrescritura.** Además de los métodos abstractos y los de `ISmartGrids` (todos con `@Override`):
  - `PlantaSolar.cuotaMinima()` redefine la de `PlantaRenovable` y la reutiliza con `super.cuotaMinima()`.
  - `PlantaEolica` y `PlantaTermica` redefinen `validarRegistro(registro)` de `Planta` para exigir su tipo de registro.
  - `datosEspecificos()` de cada planta concreta extiende el de su categoría con `super.datosEspecificos()`.
  - `toString()` está redefinido en `SmartGrids`, `Planta`, `PeriodoInactividad`, `RegistroMonitoreo`, `RegistroEolica` y `RegistroTermica` (los registros extienden el de su superclase con `super.toString()`).
- **Sobrecarga.** `SmartGrids` tiene tres versiones de `registrarMonitoreo`, como pide la interfaz: con energía e incidencias, con además la velocidad del viento (crea un `RegistroEolica`) y con además el CO2 y el combustible (crea un `RegistroTermica`).
- **Polimorfismo.** `SmartGrids` trata a todas las plantas como `Planta` y cada una responde según su tipo:
  - `capacidadMinimaCuota` llama a `cuotaMinima()` y el histograma a `cumpleCuota(registro)`, sin preguntar el tipo de planta.
  - `Planta.registrarMonitoreo` llama a `validarRegistro(registro)`, que se resuelve según el tipo de planta.
  - `toString()` de `Planta` usa `tipo()` y `datosEspecificos()`, que define cada subclase.
  - El mapa de registros guarda `RegistroMonitoreo` de cualquier subclase, y el requerimiento 8 usa `esRenovable()` para elegir las plantas renovables.
- **StringBuilder.** `toString()` de `SmartGrids` y de `Planta`, `horasSinVelocidadMinimaPorPlantaEolica()` y `PlantaEolica.horasSinVelocidadMinima()`. Con + se crearía un `String` nuevo en cada concatenación, porque los `String` son inmutables.
- **Iteradores.** `Planta.limpiarRegistrosSinDatos()` recorre los registros con un `Iterator`, porque necesita borrar mientras recorre: `iterator.remove()` es la forma segura de hacerlo (con un foreach se produciría `ConcurrentModificationException`).
- **Foreach.** Se usa para recorrer sin modificar: las plantas en `consultarPlantas`, `plantasRenovablesPorFuente`, `plantasQueNuncaCumplieronCuota`, `plantasInactivasEn` y `toString`; los registros en `incidenciasPorHora`,

`nuncaCumplioCuota` y `horasSinVelocidadMinima`; y los períodos de inactividad en `estuvoActivaEn`.

3) IREP

Clase SmartGrids:

- `plantas` no es nulo y ningún valor es nulo.
- Para cada entrada (k, p) de `plantas`: k es igual a `p.getNombre()`. Por eso no hay dos plantas con el mismo nombre.

Clase Planta:

- `nombre` y `direccion` no son nulos ni vacíos; `capacidadMaxima` > 0; `fechaPuestaEnFuncionamiento` no es nula; `registros` y `periodosInactividad` no son nulos.
- Para cada entrada (k, r) de `registros`: k == `r.getFechaHora()`, `r.getNombrePlanta()` es igual a `nombre` y k no es anterior a `fechaPuestaEnFuncionamiento`.
- Ningún registro cae dentro de un período de inactividad: para todo período p y registro r, `p.contiene(r.getFechaHora())` es falso.
- `totalGenerado` es la suma de la energía generada de todos los registros.
- `periodosInactividad` está ordenada por `desde` y los períodos no se superponen: si i < j, el período i está cerrado y hasta(i) <= desde(j).
- Solo el último período puede estar abierto, y ningún período empieza antes de `fechaPuestaEnFuncionamiento`.
- `activa` == false si y solo si la lista de períodos no está vacía y el último está abierto.

Clase PlantaRenovable:

- El IREP de `Planta`; `fuentePrimaria` no es nula ni vacía; 0 < `eficienciaPromedio` <= 1.

Clase PlantaSolar:

- El IREP de `PlantaRenovable`; `superficieCubierta` > 0, `cantidadPaneles` > 0 y 0 < `horasRadiacionDiaria` <= 24.
- Sus registros no son `RegistroEolica` ni `RegistroTermica`.

Clase PlantaEolica:

- El IREP de `PlantaRenovable`; `cantidadAerogeneradores` > 0, `alturaTorres` > 0 y `velocidadMinimaViento` > 0.
- Todos sus registros son `RegistroEolica`.

Clase PlantaHidraulica:

- El IREP de `PlantaRenovable`; `cantidadTurbinas` > 0 y `capacidadEmbalse` > 0.

- Sus registros no son `RegistroEolica` ni `RegistroTermica`.

Clase PlantaNoRenovable:

- El IREP de `Planta`; `tipoCombustible` no es nulo ni vacío; `objetivoEmisionesCo2PorHora` > 0; `capacidadAlmacenamientoCombustible` > 0.

Clase PlantaTermica:

- El IREP de `PlantaNoRenovable`; todos sus registros son `RegistroTermica`.

Clase PeriodoInactividad:

- `desde` no es nulo.
- `hasta` es nulo (período abierto) o `desde` < `hasta`.

Clase RegistroMonitoreo:

- `nombrePlanta` no es nulo ni vacío.
- `fechaHora` no es nula y es una hora exacta (minutos, segundos y nanosegundos en 0).
- `energiaGenerada` >= 0 e `incidencias` >= 0.

Clase RegistroEolica:

- El IREP de `RegistroMonitoreo`; `velocidadVientoPromedio` >= 0.

Clase RegistroTermica:

- El IREP de `RegistroMonitoreo`; `co2Emitido` >= 0 y `combustibleUtilizado` >= 0.

4) Representación de SmartGrids (toString)

`SmartGrids.toString()` muestra la cantidad de plantas y, para cada una, el `toString()` de `Planta`: su detalle (nombre, estado y puesta en funcionamiento) y su tipo, la dirección, la capacidad máxima, la cuota por hora, los datos propios de su tipo, la cantidad de registros, el total generado y, si los tiene, sus períodos de inactividad. Por ejemplo, al final del código cliente se ve:

```
SmartGrids - 2 planta(s) registrada(s)
Solar Norte (ACTIVA): 2026-10-08T00:00:00 - Solar
    direccion: Ruta 1 | capacidad maxima: 1000.0 | cuota por hora: 333.33
    fuente: Sol | eficiencia: 0.8 | superficie: 1500.0 | paneles: 12 | horas de radiacion: 10.0
    registros: 1 | total generado: 500.0
Eolica Sur (INACTIVA): 2026-10-08T00:00:00 - Eolica
    direccion: Ruta 2 | capacidad maxima: 800.0 | cuota por hora: 600.0
    fuente: Viento | eficiencia: 0.75 | aerogeneradores: 4 | altura de torres: 80.0 | velocidad minima: 5.0 m/s
    registros: 1 | total generado: 200.0
    periodos de inactividad: [2026-10-08T12:00:00 -> (sigue inactiva)]
```

5) Complejidad del histograma de cumplimiento (requerimiento 7)

Llamamos D a la cantidad de días entre `fechaDesde` y `fechaHasta`, ambas inclusive (D >= 1, porque se exige que `fechaDesde` no sea posterior a `fechaHasta`), R a la cantidad de registros de la planta y P a la cantidad de plantas. Como vimos en la materia, tomamos `get` de `HashMap` y `LinkedHashMap` como O(1). Usamos las reglas del Álgebra de Órdenes: O(f) + O(g) = O(max(f, g)), n · O(f) = O(n · f) y, para una constante c, O(c) = O(1).

SmartGrids.histogramaCumplimiento y auxiliares

```
@Override
public int[] histogramaCumplimiento(String nombre, String fechaDesde, String fechaHasta) {
    Planta planta = buscarPlanta(nombre);
    LocalDate desde = parsearFecha(fechaDesde);
    LocalDate hasta = parsearFecha(fechaHasta);
    if (desde.isAfter(hasta)) {
        throw new IllegalArgumentException("La fecha desde (" + desde + ") es posterior a
hasta (" + hasta + ")");
    }
    return planta.histogramaCumplimiento(desde, hasta);
}

private Planta buscarPlanta(String nombre) {
    Planta planta = plantas.get(nombre);
    if (planta == null) {
        throw new IllegalArgumentException("No existe una planta llamada " + nombre);
    }
    return planta;
}

private static LocalDate parsearFecha(String fecha) {
    if (fecha == null) {
        throw new IllegalArgumentException("Falta la fecha");
    }
    try {
        return LocalDate.parse(fecha);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException("Fecha invalida: " + fecha + " (formato esperado
yyyy-MM-dd)");
    }
}
```

- `buscarPlanta`: un `get` del `LinkedHashMap` O(1) y una comparación O(1). T = O(1) + O(1) = O(1).
- `parsearFecha`: la comparación con `null` es O(1) y `LocalDate.parse` procesa un texto de largo fijo (`yyyy-MM-dd`), O(1). T = O(1).
- `histogramaCumplimiento` de `SmartGrids`: `buscarPlanta` O(1), dos `parsearFecha` O(1), la comparación `isAfter` O(1) y la llamada a `Planta.histogramaCumplimiento`, que se analiza a continuación.

Planta.histogramaCumplimiento

```
public int[] histogramaCumplimiento(LocalDate desde, LocalDate hasta) {
    int[] histograma = new int[24];
    for (LocalDate dia = desde; !dia.isAfter(hasta); dia = dia.plusDays(1)) {
        for (int hora = 0; hora < 24; hora++) {
            RegistroMonitoreo registro = registros.get(dia.atTime(hora, 0));
```

```
            if (registro != null && cumpleCuota(registro)) {
                histograma[hora]++;
            }
        }
    }
    return histograma;
}
```

- Crear el arreglo `new int[24]` inicializa 24 posiciones: O(24) = O(1).
- Cuerpo del `for` interno: `dia.atTime(hora, 0)` O(1), `registros.get(...)` O(1), `cumpleCuota(registro)` O(1) (ver abajo) y el incremento O(1). O(1) + O(1) + O(1) + O(1) = O(1).
- `for` interno: 24 iteraciones de costo O(1), más la comparación y el incremento de `hora`: 24 · O(1) = O(24) = O(1).
- `for` externo: una iteración por día, D iteraciones. Cada una ejecuta el `for` interno O(1), la comparación `isAfter` O(1) y `plusDays` O(1): D · (O(1) + O(1) + O(1)) = D · O(1) = O(D).
- El `return` es O(1). T = O(1) + O(D) + O(1) = O(max(1, D, 1)) = O(D).

Cumplimiento de la cuota según el tipo de planta

`cumpleCuota` es polimórfico; el histograma ejecuta la versión que corresponde al tipo de planta:

```
@Override
public double cuotaMinima() {
    return getCapacidadMaxima() * eficienciaPromedio;
}

@Override
public boolean cumpleCuota(RegistroMonitoreo registro) {
    return registro.getEnergiaGenerada() >= cuotaMinima();
}


@Override
public double cuotaMinima() {
    return super.cuotaMinima() * horasRadiacionDiaria / 24;
}


@Override
public double cuotaMinima() {
    return getCapacidadMaxima() / 2;
}

@Override
public boolean cumpleCuota(RegistroMonitoreo registro) {
    RegistroTermica termico = (RegistroTermica) registro;
    return termico.getEnergiaGenerada() >= cuotaMinima()
            && termico.getCo2Emitido() < getObjetivoEmisionesCo2PorHora();
}
```

- Todas las versiones de `cuotaMinima` hacen un número fijo de operaciones aritméticas y llamadas a getters: O(1). `cumpleCuota` hace a lo sumo un casteo, dos comparaciones, una llamada a `cuotaMinima` y dos getters: O(1) + O(1) + O(1) = O(1), cualquiera sea el tipo de planta.

Conclusión

T(histogramaCumplimiento) = O(1) + O(1) + O(1) + O(1) + O(D) = O(max(1, D)) = **O(D)**.
El costo depende solo de la cantidad de días consultados: no depende de la cantidad de registros R de la planta ni de la cantidad de plantas P. Lo logramos porque los registros están en un `HashMap` cuya clave es la fecha y hora: en lugar de recorrer todos los registros y filtrar los del rango, lo que costaría O(R), se busca directamente el registro de cada una de las 24 horas de cada día. Una planta con años de registros responde igual de rápido que una nueva cuando se consulta un rango corto.

6) Tests propios

Además de `SmartGridsTest`, entregamos `SmartGridsPropioTest` con casos borde. Ambos pasan, y el código cliente se ejecuta sin errores. Los tests propios verifican, entre otras cosas:

- que se rechacen los datos inválidos y los valores negativos al registrar plantas y monitoreos;
- que las consultas sobre una planta inexistente fallen;
- que no se acepten registros que caen en un período de inactividad pasado y que `plantasInactivasEn` use el historial;
- que activar o desactivar dos veces no cambie nada y que no se pueda reactivar antes de desactivar;
- que cada planta acepte solo su tipo de registro y que el registro corresponda a la hora completa;
- que el histograma ignore registros fuera del rango y acepte un rango de un solo día;
- los formatos de los requerimientos 1.b, 8, 10 y 12, la limpieza sin registros vacíos y el `toString`.